Karadeniz Teknik Üniversitesi
Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Hızlı Başvuru Kağıdı #4: Qt Framework'üne Giriş

Yazılım Geliştirme II – Hafta 4

Qt Ekosistemi ve CMake

Qt Modülleri

Modül	Açıklama
`Qt::Core`	Temel veri türleri, olay döngüsü
`Qt::Widgets`	Masaüstü arayüz bileşenleri
`Qt::Gui`	Pencere, resim, font yönetimi
`Qt::Network`	TCP/UDP soket, HTTP
`Qt::Sql`	Veritabanı erişimi
`Qt::Concurrent`	Çok kanallı işlemler

CMakeLists.txt Temel Yapı

```
cmake_minimum_required(VERSION 3.16)
project(OBYSApp LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_AUTOMOC ON)

find_package(Qt6 REQUIRED COMPONENTS
    Widgets)

add_executable(obys_app
    main.cpp
    mainwindow.cpp
    mainwindow.h)

target_link_libraries(obys_app PRIVATE
    Qt6::Widgets)
```

CMAKE_AUTOMOC

`set(CMAKE_AUTOMOC ON)` açık olmalı! Aksi halde `Q_OBJECT` makrosu işlenmez, sinyal/slot bağlantıları çalışmaz.

İlk Qt Uygulaması

```
#include <QApplication>
#include <QLabel>

int main(int argc, char* argv[]) {
    QApplication app(argc, argv);
    QLabel etiket("Merhaba Qt!");
    etiket.show();
    return app.exec(); // olay dongusu
}
```

Derleme Adımları

```
mkdir build && cd build
cmake .. -DCMAKE_PREFIX_PATH=/Qt/6.x
cmake -build .
./obys_app
```

QObject ve Bellek Yönetimi

QObject Temel Yapısı

Tüm Qt nesnelerinin taban sınıfıdır.

Özellik	Açıklama
`Q_OBJECT`	Meta-object desteği makrosu
`objectName`	Nesneye isim atama
`parent()`	Ebeveyn nesneyi döndürür
`children()`	Çocuk listesi
`findChild()`	Ada göre çocuk bul
`deleteLater()`	Güvenli silme

Q_OBJECT ve moc

```
class OgrenciFormu : public QWidget {
    Q_OBJECT // moc icin zorunlu!
public:
    explicit OgrenciFormu(
        QWidget* parent = nullptr);
signals:
    void ogrenciEklendi(int numara);
public slots:
    void kaydet();
};
```

Parent-Child Bellek Kuralı

Ebeveyn silinince tüm çocuklar otomatik silinir. `new` ile oluşturulan widget'lara **parent** ver!

`auto* btn = new QPushButton("Kaydet", this);`
→ `this` silinince `btn` de silinir.

Kural: Parent'lı nesneye `delete` çağırma! Yığında (`stack`) oluşturulan nesnelere parent verme!

Nesne Ağacı Örneği

```
QWidget* pencere = new QWidget;
auto* duzen = new QVBoxLayout(pencere);
auto* btn = new QPushButton("Tamam",
                            pencere);
// pencere silinince -> duzen, btn
// otomatik silinir
delete pencere; // hepsini temizler
```

Sinyal/Slot ve Widget'lar

connect() Sözdizimi

```
// Yeni syntax (tur guvenli)
connect(gonderen, &Sinif::sinyal,
        alici,    &Sinif::slot);

// Lambda ile
connect(buton, &QPushButton::clicked,
    [this]() { kaydet(); });

// Parametreli sinyal
connect(spin, &QSpinBox::valueChanged,
    [](int deger) {
        qDebug() << "Yeni:" << deger;
    });
```

Bağlantı Türleri

Tür	Davranış
`Auto`	Aynı thread → Direct
`Direct`	Anında çağırır
`Queued`	Olay kuyruğuna ekler
`Unique`	Tekrar bağlanmaz

Temel Widget Sinyalleri

Widget	Önemli Sinyaller
`QPushButton`	`clicked()`, `pressed()`
`QLineEdit`	`textChanged(QString)`
	`returnPressed()`
`QComboBox`	`currentIndexChanged(int)`
`QSpinBox`	`valueChanged(int)`
`QCheckBox`	`stateChanged(int)`

Özel Sinyal Tanımlama

```
// sinyal tanimla (govdesi yok!)
signals:
    void notGuncellendi(int no, double n);

// sinyal yayinla
emit notGuncellendi(12345, 3.75);

// slot tanimla
public slots:
    void notDegisti(int no, double n) {
        qDebug() << no << ":" << n;
    }
```

Widget Oluşturma Rehberi

Metin göster → `QLabel`
Buton → `QPushButton`
Tek satır giriş → `QLineEdit`
Sayı girişi → `QSpinBox`
Seçim kutusu → `QComboBox`
Onay kutusu → `QCheckBox`